

MILESTONE 03: SCIENTIFIC EVALUATION REPORT

Polling Server & Fixed-Priority Scheduling Foundations

1. CORE ARCHITECTURAL STRENGTHS

- **Strict Temporal Isolation:** The Polling Server (PS) functions as an execution firewall. Because its capacity (C_s) and period (T_s) are strictly bounded, an unexpected surge of aperiodic requests cannot cause budget overruns or starve hard real-time periodic tasks.
- **Predictable Schedulability Analysis:** From a scheduling theory perspective, the PS behaves exactly like a standard periodic task. This allows designers to use classical Rate Monotonic Analysis (RMA) and Response Time Analysis (RTA) without introducing non-deterministic jitter into the system equations.
- **Optimal Kernel Efficiency:** The PS requires zero complex runtime timers or variable replenishment tracking. It runs with $O(1)$ scheduler complexity, resetting its budget solely during the standard periodic system timer tick.

2. CRITICAL FLAWS & OPERATIONAL RISKS

- **The "Capacity Waste" Phenomenon:** The PS lacks budget preservation rules. If it is released and finds the aperiodic request queue completely empty, it instantly discharges its remaining budget to zero ($C_s = 0$) and suspends itself until the next period.
- **Worst-Case Latency Penalty:** Due to capacity wasting, an aperiodic event arriving immediately after an empty server tick must wait an entire period (T_s) for replenishment. The absolute worst-case response time for an isolated task ($C_a \leq C_s$) is mathematically bounded by:

$$R_{max} = T_s + C_s$$

- **Resource Blocking Vulnerabilities:** Traditional analysis assumes complete task independence. If an aperiodic task blocks on a shared mutex held by a lower-priority periodic task, the server's budget continues to tick down while blocked, causing catastrophic budget starvation without progress.

3. SCALABILITY & WORKLOAD APPLICABILITY

- **The Schedulability Bottleneck:** Minimizing aperiodic response latency requires decreasing T_s . However, as T_s drops, the server's processor utilization footprint ($U_s = C_s / T_s$) climbs rapidly, heavily degrading the schedulability of lower-priority periodic tasks.

- **Suitable Workloads:** Highly applicable for non-critical background telemetry, diagnostic logging, or routine sensor commands arriving via UART/CAN where latency bounds are relaxed but protecting the primary control loop from starvation is mandatory.
- **Unsuitable Workloads:** Entirely problematic for high-frequency asynchronous network packets (e.g., Ethernet IP packets in an IoT gateway) or safety-critical reactive triggers (e.g., airbag deployment) where arrival misalignment penalties can drop packets or violate hard deadlines.

4. EXTENSIONS & ALTERNATIVE SELECTIONS

- **Deferrable Server (DS) Alternative:** Preserves its budget throughout the period to significantly reduce response latency, but introduces a scheduling penalty for lower-priority tasks due to worst-case back-to-back execution behaviors.
- **Sporadic Server (SS) Alternative:** Preserves budget and retains optimal periodic utilization bounds, but introduces heavy real-time operating system kernel memory and runtime overhead to manage dynamic replenishment timers.
- **Dynamic Sizing Extension:** A key open adaptation involves using runtime control theory or machine learning heuristics to dynamically scale C_s and T_s when aperiodic arrival patterns are non-stationary and unknown at design time.